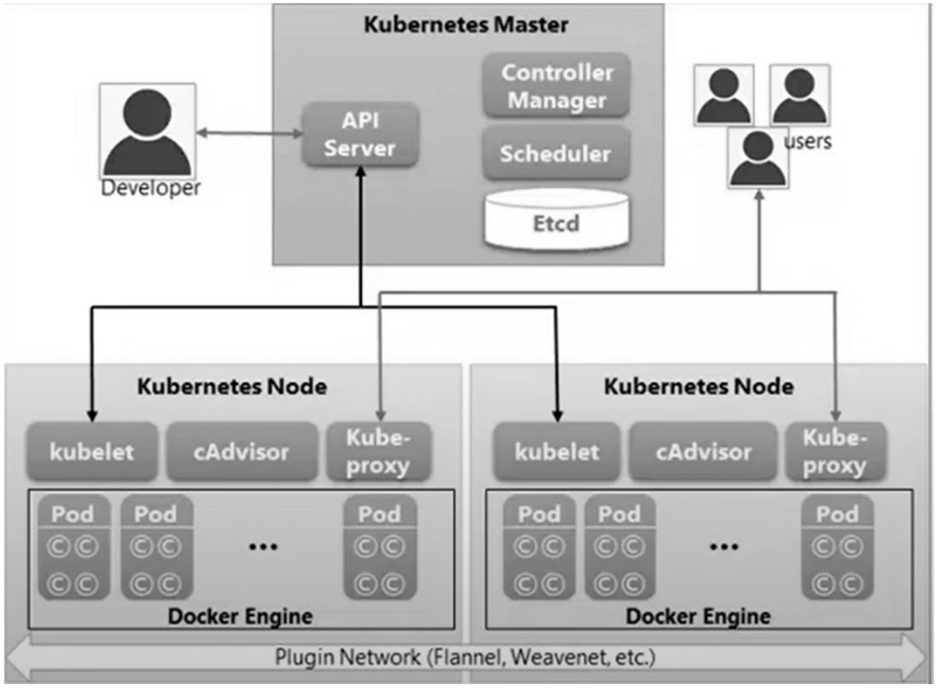
Kubernetes

Orchestration des conteneurs

Kubernetes

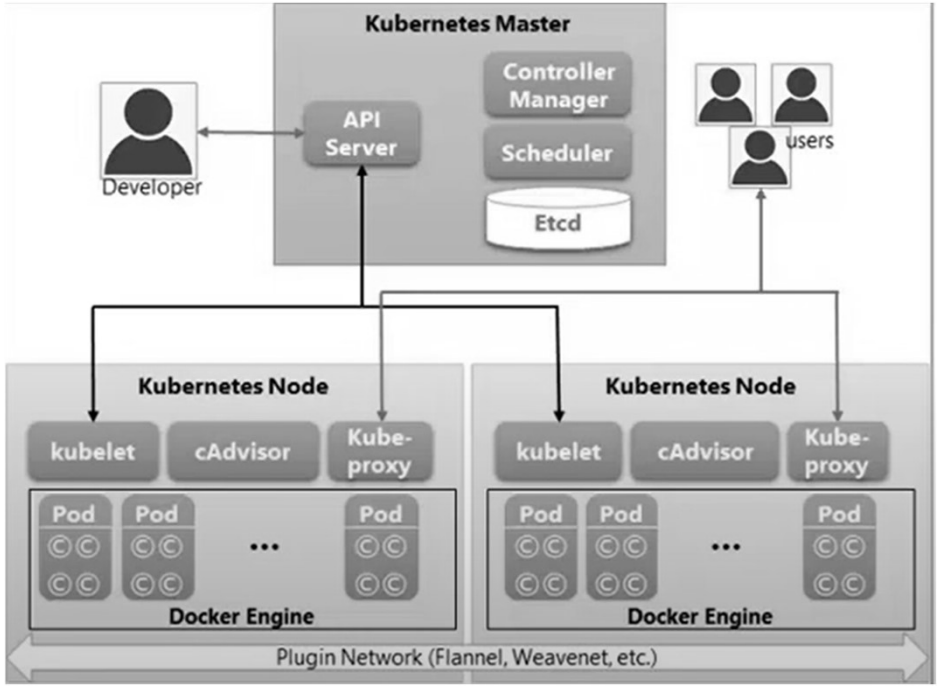
- Outils d'orcherstration des conteneurs
- Mot d'origine Grèque signifiant « Timonier », ce lui qui dirige le bateau.
- Inspiré du projet Borg dont google s'en sert depuis des années pour gérer ses conteneurs.
- Développé par google avec ses partenaires : Redhat, CoreOS, IBM, Microsoft, MesoSphere, Vmware, HP, etc.
- Ecrit en langage Go
- Peut tourner en:
  - Local, Cloud (GCE, Azure, AWS, ..), Machines Physiques
- Kubernetes REST API pour exposer ses fonctionnalités.
- L'objectif est de gérer directement des applications et non pas de machines (gestion des applications indépendamment de l'infrastructure)



Orchestration des conteneurs

Qu'est ce qu'on peut faire avec Kubernetes?

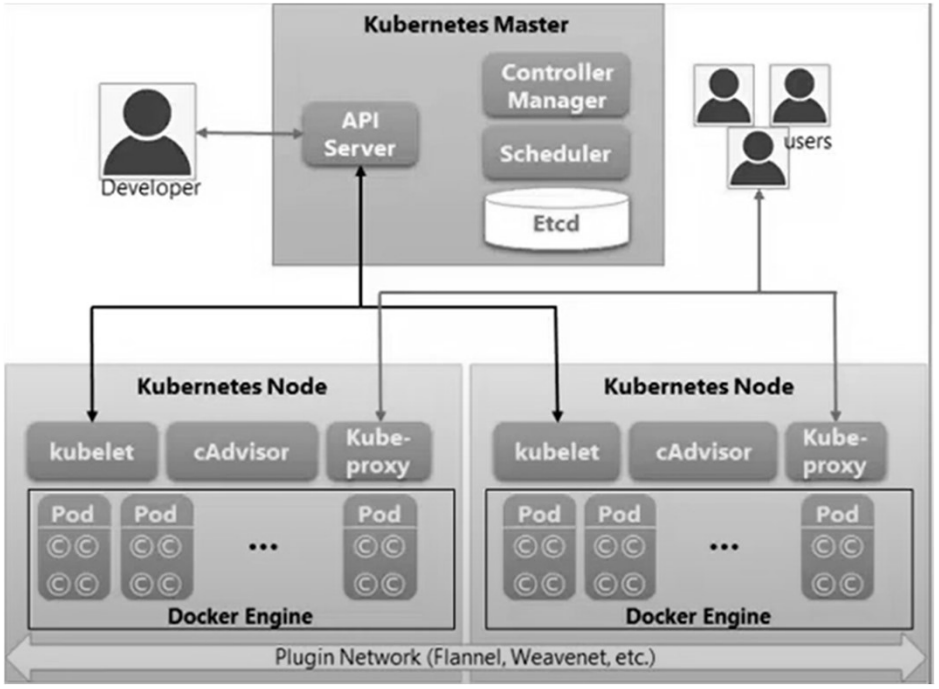
- Déployer une application sous forme de conteneurs d'une manière très rapide et prévisible
- Automatiser le déploiement et la réplication des containers.
- Organiser les conteneurs en **groupes** et faire du **Load balancing (équilibrage de charge)**
- Déclarer l'architecture cible (Nombre de pods, contenu, stratégie de mise à jour, etc.) et laisser le système atteindre et à maintenir cette cible
- Faire des mises à jour sans interrompre le service
- Séparer l'application de l'architecture sous-jacente.
- Détecter les problèmes est les résoudre tout seul



Orchestration des conteneurs

Concepts de base

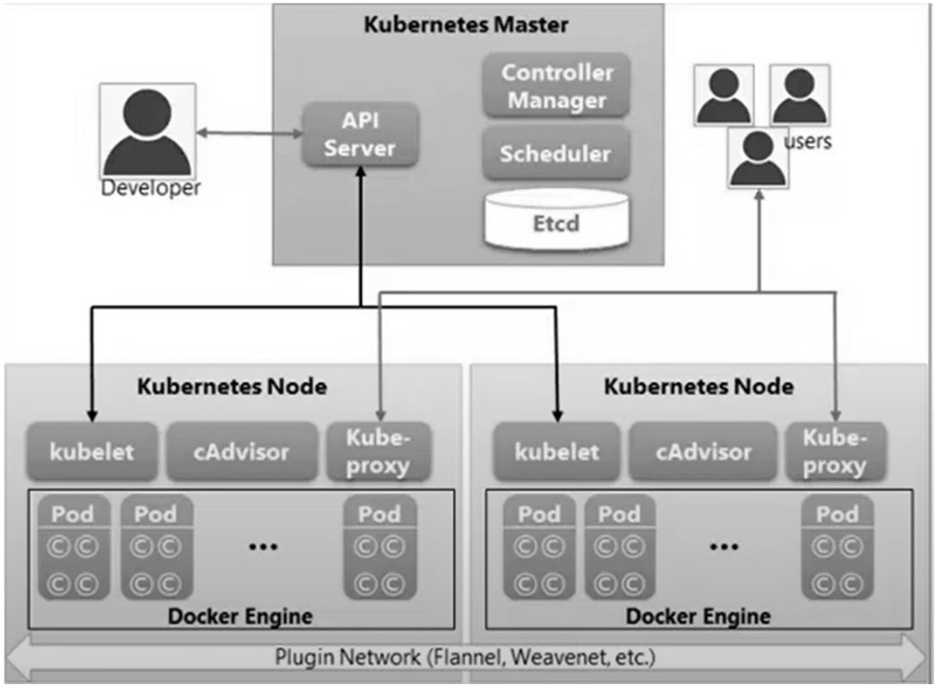
- **Cluster :** Ensemble de machines physiques ou virtuelles utilisées par Kubernetes pour faire tourner des applications.
- **Node :** Une machine physique ou virtuelle dans laquelle seront déployés des Pods. Chaque Node doit exécuter un Container Engine comme Docker.
- **Pod:** Groupe de containers déployés ensemble
- **Namespace :** Segmenter Pod, Volume, secrêts, …
- **Kubectl :** Application en ligne de commande permettant d'intérragir avec Kubernetes.



Orchestration des conteneurs

Architecture de Kubernetes

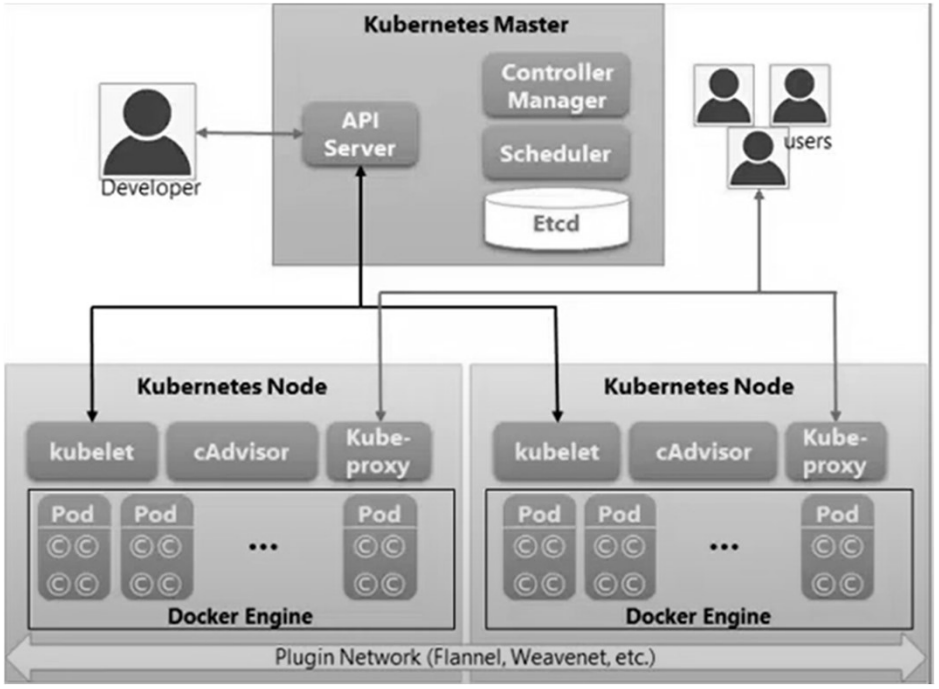
- Kubernetes suit l'architecture maitre/esclave.
- Les composants de kubernetes peuvent être divisés en ceux qui gèrent un nœud individuel et ceux qui font partie du plan de contrôle



Orchestration des conteneurs

Architecture de Kubernetes

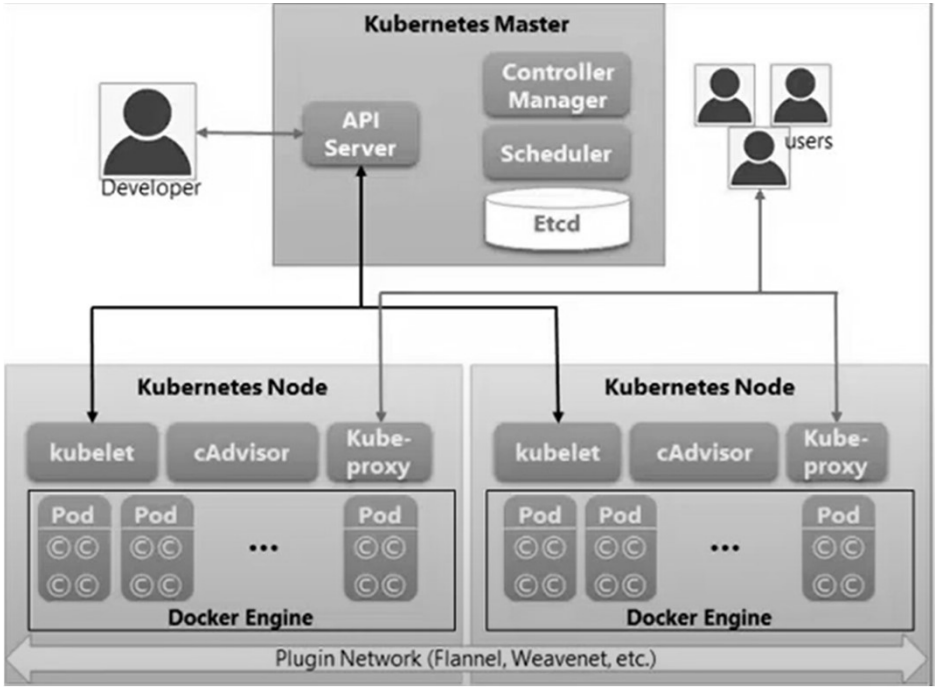
- Kubernetes Master
  - Unité de contrôle principale qui gère la charge de travail et dirige les communications dans le système.
  - Le plan de contrôle de Kubernetes consiste en plusieurs composants, chacun ayant son propre processus, qui peuvent s'exécuter sur un seul Node maître ou sur plusieurs maîtres permettant de créer des clusters haute disponibilité.



Orchestration des conteneurs

Architecture de Kubernetes

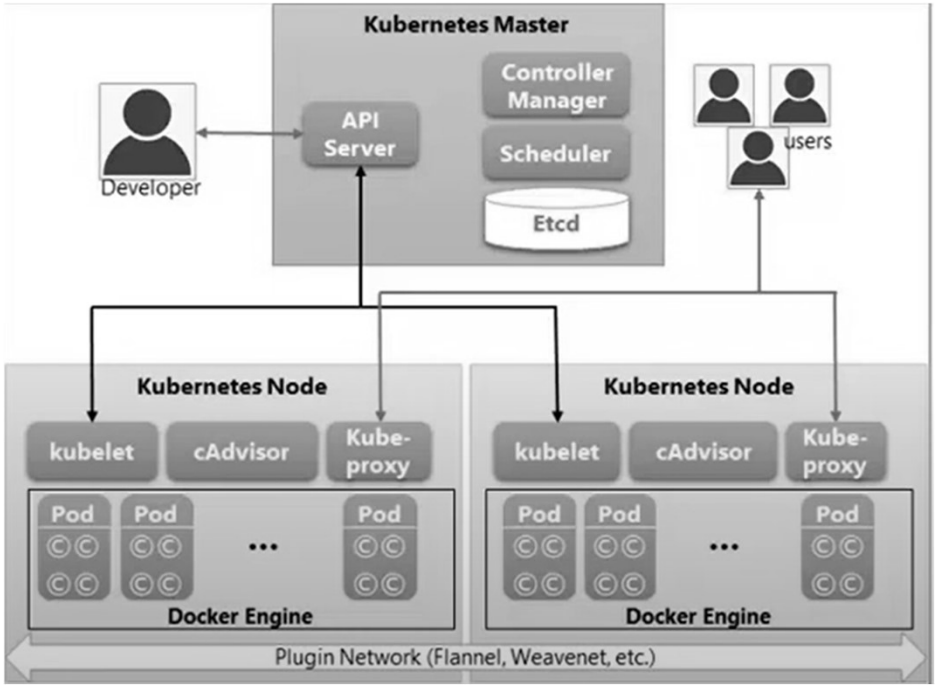
- Kubernetes Master
  - Les différents composants du plan de contrôle de Kubernetes sont décrits ci-dessous:
    - **Etcd** : Unité de stockage Distribuée persistante et légère (la configuration, état du cluster, les événements qui se produisent …)
    - **API Server**: API REST de communication avec les composants internes et externes
    - **Scheduler**: L'ordonnanceur qui permet de sélectionner quel Node devrait faire tourner un Pod (récolter en temps réel des informations sur l'état des neuds: l'occupation mémoire, capacité disque, taux d'utilisation de cpu)
    - **Controller manager** : processus dans lequel s'exécutent les contrôleurs principaux de Kubernetes tels que DaemonSet Controller (service principal) et le Replication Controller.



Orchestration des conteneurs

Architecture de Kubernetes

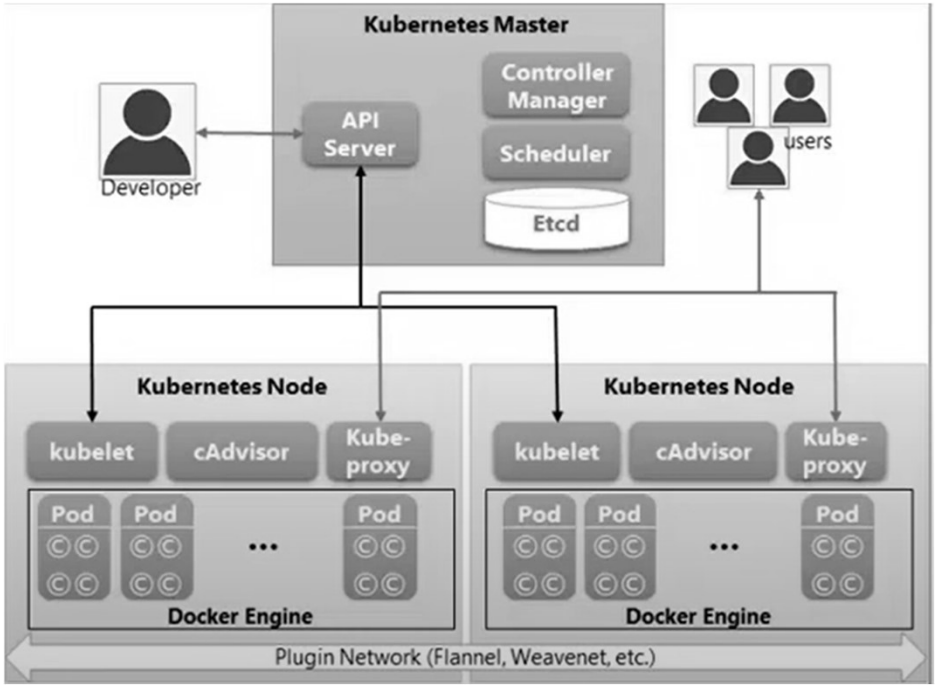
- Kubernetes Node
  - Le Node aussi appelé Worker est une machine unique (ou une machine virtuelle) où des conteneurs (charges de travail) sont déployés.
  - Chaque Node du cluster doit exécuter le programme de conteneurisation (par exemple Docker Engine), ainsi que les composants pour communiquer avec les composants du plan de contrôle du Master.



Orchestration des conteneurs

Architecture de Kubernetes

- Kubernetes Node
  - Les composants d'un Worker Node de Kubernetes sont:
    - **Kubelet** : Responsable de l'état d'exécution du nœud (c'est-à-dire, d'assurer que tous les conteneurs sur un nœud sont en bonne santé organisés en Pods sous la direction du plan de contrôle)
    - **Kube-proxy** : Responsable d'effectuer le routage du trafic vers le conteneur approprié en se basant sur l'adresse IP et le numéro de port de la requête entrante
    - **cAdvisor** : Agent qui surveille et récupère les données de consommation des ressources et des performances comme le processeur, la mémoire, ainsi que l'utilisation disque et réseau des conteneurs du Node.



Orchestration des conteneurs

Architecture de Kubernetes

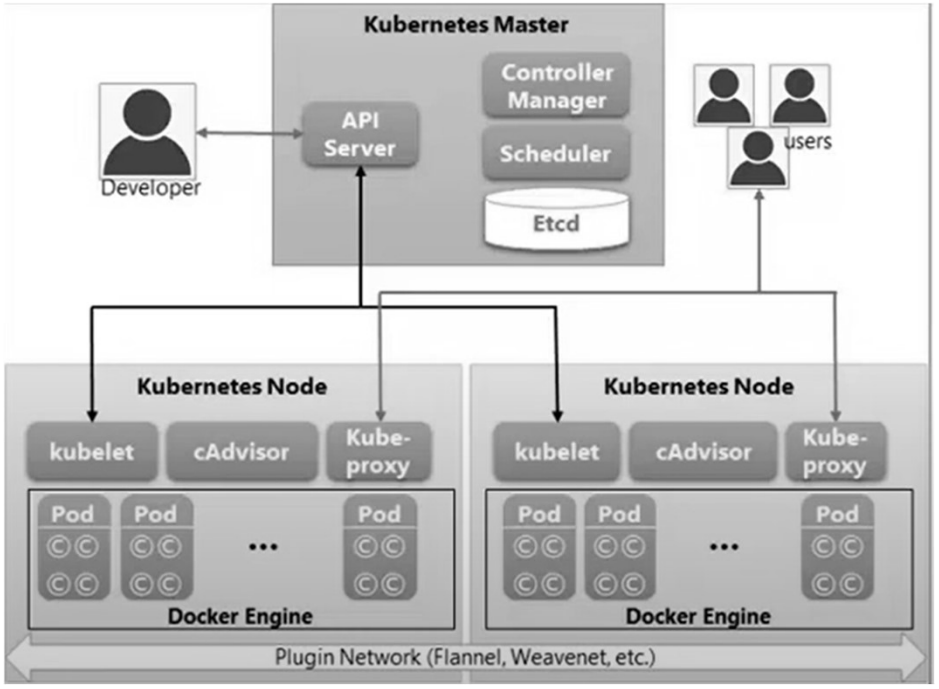
- Pod
  - Groupe de containers déployés ensemble
  - Les containers du Pod :
    - Sont toujours démarrés, arrêtés et répliqués en groupe
    - Partagent le même Nework, Namespace, IP et Ports
    - Peuvent communiquer ensemble (un conteneur peut contacter un autre sur le même pod) en utilisant localhost
    - Peuvent partager des données via des shared volumes
  - Le Pod est l'entité de base qu'on va répliquer et qu'on va mettre sur différents nœuds du cluster.

```
// Créer un pod
$ kubectl create -f mypod.yml
//Lister tous les pods
$ kubectl get pods
//Supprimer un pod
$ kubectl delete Pod_ID
```

Orchestration des conteneurs

Architecture de Kubernetes

- Volume
- Espace de stockage
  - qui peut être attaché
    - à un conteneur
    - à un groupe de conteneurs dans un même Pod,
  - Peut persister indépendamment du conteneur
  - Par exemple, il permet aux conteneurs de partager un même répertoire dans un même Pod.



Orchestration des conteneurs

Déploiement du contrôleur

- Elément central de Kubernetes
- C'est le fichier dans lequel on décrit l'architecture cible que l'on veut atteindre.
  - Les conteneurs qu'on veut mettre dans le Pod
  - Les détails de réplication
  - Les stratégies qu'on va utiliser pour les mises à jour
- C'est le déploiement du contrôleur de Kubernetes qui va se charger de maintenir cet état.
  - Par exemple, s'il y a un Pod qui plante, il va le désactiver et l'enlever du load balancer et créer un autre à coté pour démarrer sur un autre nœud pour que ça puisse continuer à fonctionner dans les mêmes conditions exigées dans le descripteur de déploiement.

myDeployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.7.9
        ports:
        - containerPort: 80
```

```
$ kubectl create -f myDeployment.yaml
```

Orchestration des conteneurs

Déploiement du contrôleur

- Dans cet exemple:
  - Un déploiement nommé nginx est crée, indiqué par le champ metadata:name
  - Le déploiement crée trois pods répliqués, indiqués par le champ replicas
  - Le modèle de pod, ou le champ spec: template, indique que ses pods sont libellés app:nginx
  - La spécification du modèle de pod, ou du champ template: spec, indique que les pods exécutent un contneur, nginx, qui exécute l'image nginx dans sa version 1.7.9
  - Le déploiement ouvre le port 80 pour une utilisation par les pods
- En résumé, le modèle de pod contient les instructions suivantes pour les pods créées par ce déploiement:
  - Chaque pod est libellé app:nginx
  - Créez un conteneur et nommez-le nginx
  - Exécutez l'image nginx dans la version 1.7.9
  - Ouvrez le port 80 pour envoyer et recevoir du trafic

myDeployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.7.9
        ports:
        - containerPort: 80
```

```
$ kubectl create -f myDeployment.yaml
```

Orchestration des conteneurs

Service

- Permet d'accéder à un groupe de Pods avec une même adresse IP statique. Un peut plus que le load balancer
  - On identifie les Pods par des labels (Selectors) qui seront les cibles du load balancer
  - Peut faire du load balancing
  - Différents Types :
    - ClusterIP: expose le service à l'intérieur du cluster via une adresse IP interne
      - Exemple : Un service de base de données accessible uniquement par d'autres applications dans le cluster.
    - NodePort: expose le service via un port spécifique sur chaque nœud du cluster
      - Exemple : Une application web simple accessible depuis votre réseau local.
    - LoadBalancer: expose le service via un load balancer externe.
      - Exemple : Une application web en production accessible publiquement avec plusieurs pods derrière le load balancer.

myDeployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.7.9
        ports:
        - containerPort: 80
```

```
$ kubectl create -f myDeployment.yaml
```

Orchestration des conteneurs

Installation de Kubernetes

- **Kubectl** est un outil en ligne de commande de kubernetes, kubectl, vous permet d'exécuter des commandes dans les clusters Kubernetes. Vous pouvez utiliser kubectl pour déployer des applications, inspecter et gérer les ressources du cluster et consulter les logs, ……
  - Installer Kubectl:

    https://kubernetes.io/docs/tasks/tools/install-kubectl/

- **Minikube**, qui est un outil qui fait tourner un cluster Kubernetes à un nœud unique dans une machine virtuelle sur votre machine. C'est une machine virtuelle qui avec l'essentiels des packages de Docker et Kubernetes
  - Installer minikube:

    https://kubernetes.io/docs/tasks/tools/install-minikube/

Orchestration des conteneurs

Installation de Kubernetes

- Pour installer minikube, il faut disposer d'un hyperviseur de virtualisation
- Si la version de windows que vous avez ne dispose pas de l'hypervisuer Hyper-V, il est recommandé d'installer VirtualBox.
- Pour vérifier si votre système supporte la virtualisation utiliser la commande syteminfo.

```
> systeminfo
...
Hyper-V Requirements:    VM Monitor Mode Extensions: Yes
                         Virtualization Enabled In Firmware: Yes
                         Second Level Address Translation: Yes
                         Data Execution Prevention Available: Yes
```

Système d'exploitation	Hyperviseurs supportés
macOS	VirtualBox, VMware Fusion, HyperKit
Linux	VirtualBox, KVM
Windows	VirtualBox, Hyper-V

Orchestration des conteneurs

Installation de Kubernetes

- La façon la plus simple d'installer Minikube sur windows est d'utiliser Chocolatey (exécuter avec les droits administrateur): https://chocolatey.org

```
> choco install minikube kubernetes-cli
```

- Une fois l'installation de Minikube terminée, fermez la session CLI en cours et redémarrez. Minikube devrait avoir été ajouté à votre path automatiquement
- Une façon de faire c'est de télécharger directement l'exécutable de Minikube du lien suivant:

https://github.com/kubernetes/minikube/releases/latest

- Cette opération téléchargera une image de la machine virtuelle légère (Boot2Docker) qui contient tous les outils docker et kubernetes, ensuite elle crée une configuration de démarrage et démarre le cluster kubernetes

- Une fois l'installation est terminée, exécuter la commande:

```
C:\Users\med>minikube start
* minikube v1.3.1 on Microsoft Windows 10 Home 10.0.17134 Build 17134
* Tip: Use 'minikube start -p <name>' to create a new cluster, or 'minikube delete' to delete this one.
* Starting existing virtualbox VM for "minikube" ...
* Waiting for the host to be provisioned ...
* Preparing Kubernetes v1.15.2 on Docker 18.09.8 ...
* Relaunching Kubernetes using kubeadm ...
* Waiting for: apiserver proxy etcd scheduler controller dns
* Done! kubectl is now configured to use "minikube"
C:\Users\med>
```

Orchestration des conteneurs

Installation de Kubernetes

- Configurer l'environnement local de docker client
  - Pour connecter le client docker à Docker engine fourni par Minikube, nous avons besoin de change la variable d'environnement pour qu'elle pointe vers le host de la machine virtuelle Minikube.
  - La commande minikube docker-env retourne les variables d'environnement nécessaires à configurer l'environnement local pour utiliser le Docker daemon à l'intérieur de Minikube.

```
$ echo $(minikube docker-env)

export DOCKER_TLS_VERIFY="1" export DOCKER_HOST="tcp://192.168.99.100:2376" export
DOCKER_CERT_PATH="C:\Users\med\.minikube\certs" # Run this command to configure your
shell: # eval $(minikube docker-env)
```

```
$ eval $(minikube docker-env)

$ echo $DOCKER_HOST

tcp://192.168.99.100:2376
```

Orchestration des conteneurs

Installation de Kubernetes

- Cet exemple vous montre comment exécuter une simple application Hello wolrd Node.js sur Kubernetes en utilisant Minikube.
- L'image de conteneur est construite à partir des fichiers suivants:

Dockerfile

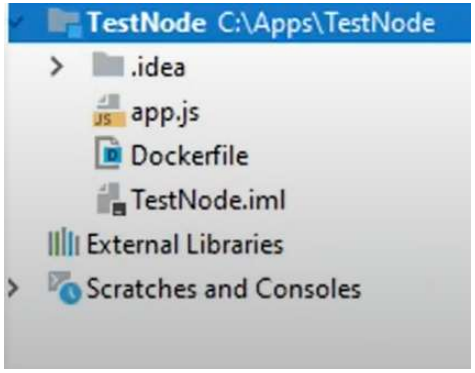
```
FROM node:6.14.2
EXPOSE 8080
COPY server.js .
CMD node server.js
```

Server.js

```
var http = require('http');

var handleRequest = function(request, response) {
    console.log('Received request for URL: ' + request.url);
    response.writeHead(200);
    response.end('Hello World!');
};
var www = http.createServer(handleRequest);
www.listen(8080);
```

https://kubernetes.io/fr/docs/tutorials/kubernetes-basics/create-cluster/cluster-interactive/



Orchestration des conteneurs

Installation de Kubernetes

- Construire l'image docker de l'application

```
$ cd c:
med@DESKTOP-PJCKK8F MINGW64 /c
$ cd Apps/TestNode
med@DESKTOP-PJCKK8F MINGW64
/c/Apps/TestNode
$ ls
app.js  Dockerfile  TestNode.iml
```

```
$ docker build . -t app-test:v1
Sending build context to Docker daemon  15.87kB
Step 1/4 : FROM node:6.14.2
 ---> 00165cd5d0c0
Step 2/4 : EXPOSE 8080
 ---> Using cache
 ---> 7930cc8e09d1
Step 3/4 : COPY app.js .
 ---> Using cache
 ---> b352977ee20b
Step 4/4 : CMD node app.js
 ---> Using cache
 ---> 5a5e5b3c1675
Successfully built 5a5e5b3c1675
Successfully tagged app-test:v1
SECURITY WARNING: You are building a Docker image from
Windows against a non-Windows Docker host. All files and
directories added to build context will have '-rwxr-xr-x'
permissions. It is recommended to double check and reset
permissions for sensitive files and directories.
```

Orchestration des conteneurs

Installation de Kubernetes

- Déployer un conteneur de app-test dans un pod kubernetes

```
$ kubectl create deployment app-test --image=app-test:v1
deployment.apps/app-test created

$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
app-test            1/1     1            1           14s
hello-minikube      1/1     1            1           3h3m
kubernetes-bootcamp 1/1     1            1           127m

$ kubectl get pods
NAME                                 READY   STATUS    RESTARTS   AGE
app-test-5f78dcf7f8-2ts2w            1/1     Running   0          34s
hello-minikube-6fb6cb79cc-pnmg8      1/1     Running   1          3h4m
kubernetes-bootcamp-75bccb7d87-ltgv5 1/1     Running   1          127m
```

Orchestration des conteneurs

Installation de Kubernetes

- Exposer le port 8080 de l'application pour un accès au pod de l'extérieur

```
$ kubectl expose deployment app-test --type=NodePort --port=8080
service/app-test exposed

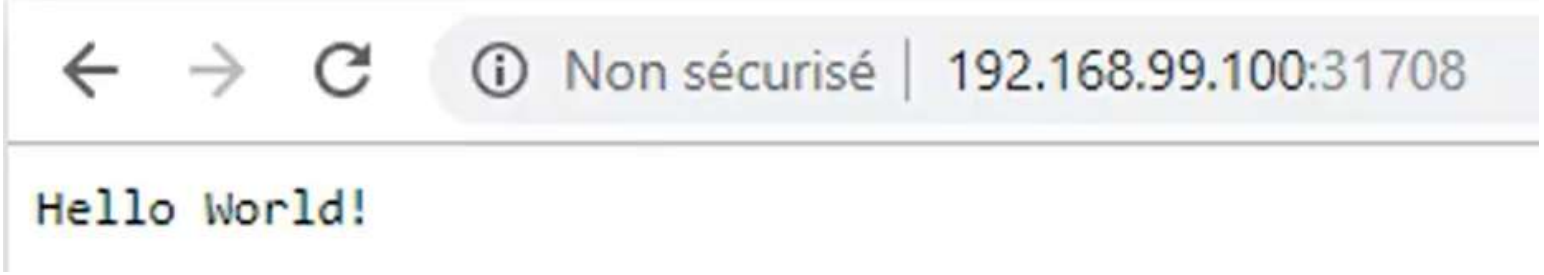
$ kubectl describe service app-test
Name:                     app-test
Namespace:                default
Labels:                   app=app-test
Annotations:              <none>
Selector:                 app=app-test
Type:                     NodePort
IP:                       10.108.16.123
Port:                     <unset>  8080/TCP
TargetPort:               8080/TCP
NodePort:                 <unset>  31708/TCP
Endpoints:                172.17.0.11:8080
Session Affinity:         None
External Traffic Policy:  Cluster
Events:                   <none>
```

Orchestration des conteneurs

Installation de Kubernetes

- Consulter le service app-test à distance

```
$ minikube service app-test
|-----------|----------|-----------------------------|
| NAMESPACE |   NAME   |             URL             |
|-----------|----------|-----------------------------|
| default   | app-test | http://192.168.99.100:31708 |
|-----------|----------|-----------------------------|
* Opening kubernetes service  default/app-test in default browser...
```



Orchestration des conteneurs

Installation de Kubernetes

- Consulter le service app-test en local

```
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
app-test-5f78dcf7f8-2ts2w           1/1     Running   0          4m40s
hello-minikube-6fb6cb79cc-pnmg8     1/1     Running   1          3h8m
kubernetes-bootcamp-75bccb7d87-ltgv5   1/1     Running   1          131m
```

```
$ winpty kubectl exec -ti app-test-5f78dcf7f8-2ts2w bash
root@app-test-5f78dcf7f8-2ts2w:/# ls
app.js  boot  etc   lib    media  opt   root  sbin  sys  usr
bin     dev   home  lib64  mnt    proc  run   srv   tmp  var
root@app-test-5f78dcf7f8-2ts2w:/# curl http://localhost:8080
Hello World!root@app-test-5f78dcf7f8-2ts2w:/# ^C
```

Orchestration des conteneurs

Installation de Kubernetes

- Consulter le service app-test en local

```
root@app-test-5f78dcf7f8-2ts2w:/# cat app.js
var http = require('http');

var handleRequest = function(request, response) {
    console.log('Received request for URL: ' + request.url);
    response.writeHead(200);
    response.end('Hello World!');
};
var www = http.createServer(handleRequest);
www.listen(8080);
```

Orchestration des conteneurs

Installation de Kubernetes

- Nettoyage

```
$ kubectl delete service app-test
service "app-test" deleted

$ minikube stop
* Stopping "minikube" in virtualbox ...
* "minikube" stopped.

$ minikube delete
* Deleting "minikube" in virtualbox ...
* The "minikube" cluster has been deleted.
```